

csiro-biomass

9位Solution

Rank	9
Team	ghatotkachh
Author	ghatotkachh
Topic ID	671026
Version	Japanese Translation

Source: <https://www.kaggle.com/competitions/csiro-biomass/discussion/671026>

Retrieved with Kaggle CLI on 2026-07-18.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

私たちのsolutionは、すべてDINOv3 ViT-Huge backboneとLocalMamba fusion blockを使う、**3 model variant**のweighted ensembleです。難しい Dry_Dead_g targetへratio-based predictionとmulti-head architecture variationで対処したことが、特に有効でした。

最終ensemble：3 model architecture × 4 fold

Model architecture

共通component

すべてのmodelは次のcore architectureを共有します。

```
Input: Image (2000×1000) → Split into Left (1000×1000) + Right (1000×1000)
↓
DINOv3 ViT-Huge Backbone (vit_huge_plus_patch16_dinov3.lvd1689m)
- Pretrained weights from timm
- Gradient checkpointing enabled for memory efficiency
- global_pool='' to retain patch tokens
↓
Feature Concatenation: [Left Tokens, Right Tokens] → (B, 2N, D)
↓
LocalMamba Fusion (2 blocks)
- LayerNorm → Gated Linear → DepthwiseConv1d → Projection
- Residual connections
↓
Adaptive Average Pooling → (B, D)
↓
Multi-Head Regression with Softplus activation (non-negative outputs)
```

Model 1: Dead予測の改善（ratio-based）

重要な工夫: Dry_Dead_g を Dry_Total_g に対する比率として予測します。

Head	出力	Activation
head_green	Dry_Green_g	Softplus
head_clover	Dry_Clover_g	Softplus
head_total	Dry_Total_g	Softplus
head_dead_ratio	Dead/Total ratio	Sigmoid [0,1]

導出:

- $\text{Dry_Dead_g} = \text{head_dead_ratio} \times \text{Dry_Total_g}$
- $\text{GDM_g} = \text{Dry_Green_g} + \text{Dry_Clover_g}$

特別な処理:

- learnable biasによるState-specific Dead調整（WAではDead $\approx$ 0）
- auxiliary ratio lossを含むmulti-task loss
- Deadに対するscale-invariant log-space loss

Training config:

- Image size: 1024 $\times$ 1024
- Loss weight: [0.12, **0.25**, 0.12, 0.16, 0.35]（Deadのweightを増加）

Model 2: 標準multi-head

Architecture: 直接予測と導出target。

Head	直接予測	導出
head_green	Dry_Green_g	—
head_dead	Dry_Dead_g	—
head_clover	Dry_Clover_g	—
—	GDM_g	Green + Clover
—	Dry_Total_g	GDM + Dead

Training config:

- Image size: 1024 $\times$ 1024
- Loss weight: [0.15, 0.15, 0.15, 0.2, 0.35]

Model 3: Total-first prediction

重要な違い: TotalとGDMを直接予測し、Deadを導出します。

Head	直接予測	導出
head_green	Dry_Green_g	—
head_clover	Dry_Clover_g	—
head_gdm	GDM_g	—
head_total	Dry_Total_g	—
—	Dry_Dead_g	Total - GDM

Training config:

- Image size: 896 $\times$ 896
- Loss weight: [0.15, 0.15, 0.15, 0.2, 0.35]

Training strategy

Data preprocessing

Image split: 横長画像 (2000×1000) をLeft half (0:mid) とRight half (mid:end) へ分割し、backboneで別々に処理します。

Cross-validation strategy

Discussionで誰かが提案していたCV strategyを使いました。threadを見つけられませんでしたでしたが、その方に感謝します。

StratifiedGroupKFold (4 fold) を、慎重に設計したstratificationとgroupingで使いました。

Stratification label: 難しいtargetの有無に基づきます。

```
stratify_col = f"{has_clover}_{has_dead}"
# Ensures each fold has balanced representation of:
#   - Samples WITH Clover (Dry_Clover_g > 0)
#   - Samples WITH Dead (Dry_Dead_g > 0)
```

Group key: 同じ収集sessionのsampleをgroup化してdata leakageを防ぎます。

```
group_key = f"{day}/{month}_{state}"
# Examples: "15/3_NSW", "22/6_WA", "8/11_Vic"
# Ensures samples collected on the same date in the same state stay together
```

これが重要な理由:

1. **Clover/Deadによるstratification:** Dry_Clover_g は0が多くsparseで、Dry_Dead_g は分散が大きいいため、各foldに同様の分布を持たせます。
2. **Date+Stateによるgrouping:** 同じ日、同じ場所で収集したsampleはweatherやpasture stateが似ている可能性があります。同じfoldへ保つことでcollection-day patternのmemorizationを防ぎます。

```
sgkf = StratifiedGroupKFold(n_splits=4, shuffle=True, random_state=42)
for fold, (_, val_idx) in enumerate(
    sgkf.split(df, df['stratify_col'], groups=df['group_key'])
):
    df.loc[val_idx, 'fold'] = fold
```

Augmentation (training)

```
A.Resize(IMG_SIZE, IMG_SIZE)
A.HorizontalFlip(p=0.5)
A.VerticalFlip(p=0.5)
A.RandomRotate90(p=0.5)
A.Rotate(limit=(-15, 15), p=0.3, border_mode=REFLECT)
A.ColorJitter(brightness=0.15, contrast=0.15, saturation=0.15, hue=0.05, p=0.5)
A.GaussNoise(var_limit=(5.0, 20.0), p=0.2)
A.Normalize(ImageNet_stats)
```

Hyperparameter

Parameter	値
Batch size	2
Gradient accumulation	8 (effective batch = 16)
Epoch	100 (early stopping)
Freeze epoch	10
Warmup epoch	3
LR backbone	1e-5
LR head	1e-4
Weight decay	1e-2
EMA decay	0.998
Patience	15
Gradient clipping	1.0
Mixed precision	FP16

Loss function

Weighted Huber Loss ($\beta=5.0$) :

```
loss =  $\sum(\text{weight}_i \times \text{SmoothL1Loss}(\text{pred}_i, \text{target}_i, \text{beta}=5.0))$ 
```

Model 1の追加loss:

- Dead/Total ratioに対するauxiliary MSE loss (weight=0.15)
- Deadに対するscale-invariant log-space loss (weight=0.05)

Inferenceとensemble

Validation TTA

validationでは4-way TTAを使いました。

1. Original
2. Horizontal Flip
3. Vertical Flip
4. Horizontal + Vertical Flip

Ensemble strategy

Fold-specific weight付きweighted average:

```
weight = {
    fold_0: 1.25,
    fold_1: 0.75,
    fold_2: 1.25,
    fold_3: 0.75
}
ensemble =  $\Sigma(\text{weight} \times \text{fold\_predictions}) / \Sigma(\text{weights})$ 
```

Post-processing

```
if target_name == "Dry_Clover_g":
    prediction *= 0.8
elif target_name == "Dry_Dead_g":
    if prediction > 20:
        prediction *= 1.1
    elif prediction < 10:
        prediction *= 0.9
```

重要な洞察

1. **Dead predictionは難しい:** Dry_Dead_g は分散が最も大きく、他featureとの相関が最も低いです。ratio-based prediction (Dead/Total) が安定性を大きく改善しました。
2. **StateはDeadに重要:** WA sampleでは一貫してDead=0です。learnable state-specific biasがこのpatternの学習に役立ちました。
3. **Resolution trade-off:** 1024×1024が僅かに良い一方、896×896は高速でした。両方を組み合わせることが有効でした。実際、896で学習したmodelも896ではなく1024で推論した方が良い結果でした。

結果

Model / Ensemble	Public score	Private score
Model 1: Ratio-Based Dead	0.76760	0.65329
Model 2: Standard Multi-Head	0.77157	0.64714
Model 3: 4-Heads (Total-First)	0.76685	0.64803
Ensemble (fold weight+post-processing)	0.77142	0.65179
Ensemble (fold weightなし、post-processingなし)	0.76772	0.64855

うまくいかなかったこと

- Metadata conditioning (State/Season embedding) は効果なし

- TTAは9時間を超えたためensembleのsubmissionでは使わなかった
- 1024を超えるimage resolution
- 2を超えるMamba block